

Tổng Hợp Kiến Thức

Khái niệm cơ bản

- **Global State:** Dữ liệu tập trung, có thể truy cập từ mọi nơi trong ứng dụng, được quản lý bởi Redux hoặc Context API.
- **Prop Drilling:** Truyền dữ liệu qua nhiều lớp component bằng props, gây rắc rối trong cây component sâu.
- **Context API:** Cơ chế tích hợp sẵn trong React để chia sẻ dữ liệu mà không cần prop drilling, phù hợp cho trạng thái tĩnh hoặc đơn giản.
- **Redux Toolkit:** Thư viện bên ngoài để quản lý trạng thái một cách dễ dự đoán, thích hợp cho trạng thái phức tạp, động và cần gỡ lỗi nâng cao.

Bảng so sánh

Tiêu chí	React Context API	Redux Toolkit
Độ phức tạp & Cài đặt	Thấp. Tích hợp sẵn trong React, API đơn giản (<code>createContext</code> , <code>useContext</code>).	Cao hơn. Cần cài <code>@reduxjs/toolkit</code> , <code>react-redux</code> , cấu hình store/slice.
Hiệu năng	Có vấn đề tiềm ẩn: tất cả component dùng context re-render khi context thay đổi.	Tối ưu: <code>useSelector</code> chỉ re-render component liên quan đến state thay đổi.
Gỡ lỗi	Khó khăn, thủ công, dùng <code>console.log</code> để theo dõi.	Mạnh mẽ: Redux DevTools theo dõi action, state tree, hiệu năng.
Xử lý bất đồng bộ	Tự xử lý (e.g., <code>useEffect</code> + <code>fetch</code>), dễ làm component chồng kênh.	Hỗ trợ tốt: <code>redux-thunk</code> hoặc RTK Query xử lý bất đồng bộ có tổ chức.
Boilerplate Code	Ít. Chỉ cần context và provider.	Nhiều hơn: cần slice, store, provider (nhưng RTK giảm nhiều so với Redux gốc).
Khả năng mở rộng	Kém hơn: khó quản lý khi ứng dụng lớn, nhiều context lồng nhau.	Tốt: cấu trúc "feature folder" với slice giúp dễ quản lý code lớn.

Khi nào dùng Context API trong Next.js?

- **Trạng thái ít thay đổi:** Theme (sáng/tối), thông tin người dùng (username, avatar), ngôn ngữ (i18n).
- **Trạng thái đơn giản:** Truyền giá trị hoặc object đơn giản qua cây component.
- **Ứng dụng nhỏ/độc lập:** Giải quyết prop drilling trong phạm vi nhỏ, tránh phức tạp hóa.
- **Lưu ý:** Tránh dùng cho trạng thái cập nhật thường xuyên (form phức tạp, giỏ hàng, thông báo real-time) vì gây vấn đề hiệu năng.

Khi nào dùng Redux Toolkit trong Next.js?

- **Trạng thái lớn, liên quan nhiều nơi:** Giỏ hàng, trình phát nhạc, bộ lọc phức tạp cần đồng bộ.
- **Logic phức tạp/bất đồng bộ:** Cần gọi API, xử lý nhiều bước (e.g., thêm sản phẩm vào giỏ hàng, cập nhật UI, thông báo).
- **Nguồn data duy nhất:** Đảm bảo luồng dữ liệu nhất quán trong dự án lớn, nhiều lập trình viên.
- **Gỡ lỗi nâng cao:** Redux DevTools giúp tìm nguyên nhân lỗi state dễ dàng.
- **Lưu ý:** Trong Next.js App Router, chỉ dùng Redux cho trạng thái client-side động. Dữ liệu server-side nên fetch trong Server Components.

Kết hợp cả hai

- **Context API:** Dùng cho trạng thái tĩnh (theme, user info).
- **Redux Toolkit:** Dùng cho trạng thái động, phức tạp (giỏ hàng, bộ lọc).
- **useState:** Trạng thái cục bộ trong component.
- **URL Search Params:** Quản lý trạng thái bộ lọc/phân trang.
- **Nguyên tắc:** Bắt đầu với `useState` /props, chuyển sang Context nếu có prop drilling, dùng Redux khi Context không đáp ứng được.

Cấu hình Redux Toolkit trong Next.js (App Router)

1. Cài đặt thư viện:

```
npm install @reduxjs/toolkit react-redux
```

2. Tạo Slice (`src/redux/slices/posts.slice.js`):

```
import { createSlice } from '@reduxjs/toolkit';

const postsSlice = createSlice({
  name: 'posts',
  initialState: { list: [] },
  reducers: {
    setPosts: (state, action) => {
      state.list = action.payload;
    },
  },
});

export const { setPosts } = postsSlice.actions;
export default postsSlice.reducer;
```

3. Tạo Store (`src/redux/store.js`):

```
import { configureStore } from '@reduxjs/toolkit';
import postsReducer from '@redux/slices/posts.slice';

export const makeStore = () => {
  return configureStore({
    reducer: { posts: postsReducer },
  });
};
```

4. Tạo Hooks (`src/redux/hooks.js`):

```
import { useDispatch, useSelector, useStore } from 'react-redux';

export const useAppDispatch = useDispatch;
export const useAppSelector = useSelector;
export const useAppStore = useStore;
```

5. Tạo Store Provider (`src/app/StoreProvider.jsx`):

```
'use client';
import { useRef } from 'react';
import { Provider } from 'react-redux';
import { makeStore } from '@redux/store';

export default function StoreProvider({ children }) {
  const storeRef = useRef(null);
  if (!storeRef.current) {
    storeRef.current = makeStore();
  }
  return <Provider store={storeRef.current}>{children}</Provider>;
}
```

6. Sử dụng trong Component (`src/app/PostsList.jsx`):

```
'use client';
import { useRef } from 'react';
import { useSelector, useDispatch } from '@redux/hooks';
import { setPosts } from '@redux/slices/posts.slice';

export default function PostsList({ initialPosts }) {
  const dispatch = useDispatch();
  const initialized = useRef(false);

  if (!initialized.current) {
    dispatch(setPosts(initialPosts));
    initialized.current = true;
  }

  const posts = useSelector((state) => state.posts.list);

  return (
    <div>
      <h2>Posts</h2>
      <ul>
        {posts.map((post) => (
          <li key={post.id}>
            <h3>{post.title}</h3>
            <p>{post.body}</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

7. Tích hợp vào Page (`src/app/page.js`):

```
import StoreProvider from './StoreProvider';
import PostsList from './PostsList';

async function getPosts() {
  const res = await fetch('<https://jsonplaceholder.typicode.com/posts>');
  if (!res.ok) throw new Error('Failed to fetch posts');
  return res.json();
}

export default async function Home() {
  const initialPosts = await getPosts();
  return (
    <StoreProvider>
      <main>
        <h1>Welcome to Posts App</h1>
        <PostsList initialPosts={initialPosts} />
      </main>
    </StoreProvider>
  );
}
```

Lưu ý

- Sử dụng `useRef` trong `StoreProvider` và `PostsList` để đảm bảo store và dữ liệu chỉ khởi tạo một lần, tránh lỗi hydration.

- Redux chỉ nên dùng cho trạng thái client-side động trong Next.js. Dữ liệu server-side nên fetch trong Server Components.
- Bắt đầu với giải pháp đơn giản (`useState` , props), sau đó dùng Context hoặc Redux khi cần thiết, tránh phức tạp hóa không cần thiết.